

1 Intrinsic multiplicative completion

Source: PrimeTensor/Completion.lean

What this file establishes

Everything so far has been about a single pair of magnitudes: how close two elements of MulRat are, measured on the ladder of PrimeTensor/Scale.lean and composed by the law of PrimeTensor/Scale/Laws.lean. This file passes to sequences. It defines what it means for a stream of magnitudes to be Cauchy for that ladder, proves that eventual agreement of two such streams is an equivalence relation, and forms the quotient

$$\text{MulReal} \equiv \{\text{Cauchy streams}\} / \{\text{asymptotic equivalence}\},$$

the carrier the rest of the development calls the multiplicative reals.

Two things have to be built before the analysis can start, and they occupy the first half of the file. A Cauchy condition speaks of a *tail* of the index set, so the depths need an order; Depth of PrimeTensor/Depth.lean carries none, and one is defined here as an inductive relation. And combining two Cauchy conditions needs a common tail, so the depths need a binary upper bound; that is join, the analogue of $\max(N_1, N_2)$.

The organising decision inherited from the whole development is that the *tolerance index and the stage index are the same type*. A classical Cauchy condition reads “for every $\varepsilon > 0$ there is an N ”: two different kinds of object, one a magnitude and one a stage. Here it reads “for every level ℓ there is an anchor a ”, and both are depths. Nothing is quantified over that is not already in the object language, and in particular no ε has to be halved — the halving of the classical proof becomes succ, which PrimeTensor/Scale/Laws.lean arranged to be exactly right.

1.1 An order on the depths

Definition 1.1 (Depth.AtOrAfter). AtOrAfter is the inductive relation on Depth generated by

$$\text{here} : \text{AtOrAfter}(n, n), \quad \text{later} : \text{AtOrAfter}(a, n) \rightarrow \text{AtOrAfter}(a, \text{succ } n).$$

Read $\text{AtOrAfter}(a, n)$ as “ n lies in the tail beginning at a ”, that is, $|a| \leq |n|$.

```
inductive AtOrAfter : Depth → Depth → Prop where
| here (n : Depth) : AtOrAfter n n
| later {anchor n : Depth} :
  AtOrAfter anchor n → AtOrAfter anchor (.succ n)
```

The recursion is on the *right* endpoint: a proof of $\text{AtOrAfter}(a, n)$ is literally the number of successor steps from a up to n , packaged as $|n| - |a|$ applications of *later* to a single *here*. Induction on such a proof is therefore induction on that difference, which is what the three lemmas below use.

Lemma 1.2 (Depth.one_atOrAfter). $\text{AtOrAfter}(\text{one}, n)$ for every n .

```
theorem one_atOrAfter : ∀ n : Depth, AtOrAfter .one n
| .one => .here .one
| .succ n => .later (one_atOrAfter n)
```

Proof. By structural recursion on n . At one the two endpoints coincide, so `here` applies; at `succ n` the recursive call supplies `AtOrAfter(one, n)` and `later` raises it. $\square$

So `one` is a least element, and the whole index set is one tail — the tail anchored at `one`. This is what lets the constant stream below choose `one` as its anchor and mean “everywhere”.

Lemma 1.3 (`Depth.succ_atOrAfter`). `AtOrAfter(a, b)` *implies* `AtOrAfter(succ a, succ b)`.

Proof. By induction on the given proof. In the `here` case both endpoints are raised to the same successor, so `here` applies again. In the `later` case the induction hypothesis gives `AtOrAfter(succ a, succ n)` and one more `later` reaches `succ(succ n)`. $\square$

Lemma 1.4 (`Depth.atOrAfter_trans`). `AtOrAfter(a, b)` *and* `AtOrAfter(b, c)` *imply* `AtOrAfter(a, c)`.

Proof. By induction on the second proof, the first being carried along unchanged. The `here` case is $b = c$, where the first proof *is* the conclusion; the `later` case appends one more step to the induction hypothesis. In effect the two step-counts are added. $\square$

Remark 1.5 (Reflexive and transitive, and nothing more is proved). `here` gives reflexivity and Lemma 1.4 gives transitivity, so `AtOrAfter` is a preorder; it is in fact the usual $\leq$ transported along $d \mapsto |d|$, hence also antisymmetric and total. Neither of those is proved, and neither is needed: a Cauchy argument only ever needs to know that a tail is nonempty, that tails shrink, and that two tails meet. Nor is `AtOrAfter` registered as a `Preorder` instance, so no order notation and no library lemma about $\leq$ is in scope — the three lemmas above are the entire theory of it available downstream.

1.2 Common tails

Definition 1.6 (`Depth.join`). `join : Depth → Depth → Depth` is defined by recursion on both arguments:

$$\text{join}(\text{one}, b) = b, \quad \text{join}(\text{succ } a, \text{one}) = \text{succ } a, \quad \text{join}(\text{succ } a, \text{succ } b) = \text{succ } \text{join}(a, b).$$

```
def join : Depth → Depth → Depth
| .one, b => b
| .succ a, .one => .succ a
| .succ a, .succ b => .succ (join a b)
```

Under $d \mapsto |d|$ this is `max`, computed by peeling one successor from each argument until one of them bottoms out.

Lemma 1.7 (`Depth.left_atOrAfter` and `right_atOrAfter`). `AtOrAfter(a, join(a, b))` *and* `AtOrAfter(b, join(a, b))` *for all* a, b .

```
theorem left_atOrAfter : ∀ a b : Depth, AtOrAfter a (join a b)
| .one, b => one_atOrAfter b
| .succ a, .one => .here (.succ a)
| .succ a, .succ b => succ_atOrAfter (left_atOrAfter a b)
```

Proof. Both are structural recursions following the three cases of Definition 1.6. For the left bound: when a is **one** the join is b and Lemma 1.2 applies; when b is **one** the join is a itself and **here** applies; in the successor case the recursive call is raised by Lemma 1.3. The right bound is the same recursion with the two degenerate cases exchanged — the join of **one** with b is b , so **here** serves, and the join of $\text{succ } a$ with **one** is $\text{succ } a$, reached by Lemma 1.2. $\square$

Remark 1.8 (An upper bound, not the least one). Only the two bounds are proved. **join** is not shown commutative, associative or idempotent, and it is not shown to be a least upper bound; it is used exclusively through Lemma 1.7, where all that matters is that some single anchor lies in both tails. Anything stronger would be true and unused.

1.3 Cauchy streams

Definition 1.9 (**MulStream** and **IsMulCauchy**). A *stream* is a function $s : \text{Depth} \rightarrow \text{MulRat}$, and s is *multiplicatively Cauchy* when

$$\forall \ell \exists a \forall m, n : \text{AtOrAfter}(a, m) \wedge \text{AtOrAfter}(a, n) \longrightarrow \text{ScaleWithin}(\ell, s_m, s_n).$$

```
def IsMulCauchy (s : MulStream) : Prop :=
  ∀ level : Depth,
    ∃ anchor : Depth,
      ∀ m n : Depth,
        Depth.AtOrAfter anchor m →
        Depth.AtOrAfter anchor n →
        MulRat.ScaleWithin level (s m) (s n)
```

Design note 1.10 (One type for both indices). The classical condition quantifies over a tolerance $\varepsilon > 0$ drawn from the codomain and produces a stage N drawn from the index set. Here both the level ℓ and the anchor a are depths, and the codomain is not quantified over at all. Three consequences follow, and all three are used below.

The stream is indexed from **one**, so there is no stage zero and no convention about whether the first term counts; a tail is never empty, since Lemma 1.2 makes the whole index set the tail at **one**.

The levels are not quantified over an ordered field, so the classical move of replacing ε by $\varepsilon/2$ is unavailable — and unnecessary. Its replacement is $\ell \mapsto \text{succ } \ell$, which exists because **Depth** is closed under successors, and which does the same work because **PrimeTensor/Scale/Laws.lean** proved that two steps at $\text{succ } \ell$ compose into one at ℓ . Halving is thereby replaced by a constructor.

Finally, closeness is **ScaleWithin**, which is symmetric but not transitive at a fixed level, so the definition cannot be simplified to consecutive terms. The condition genuinely quantifies over both m and n in the tail.

Definition 1.11 (**MulCauchyStream**). **MulCauchyStream** is the structure bundling a stream term with a proof *cauchy* that it satisfies Definition 1.9.

Lemma 1.12 (**MulCauchyStream.constant** and **constant_term**). *For $q : \text{MulRat}$ the stream constantly equal to q is Cauchy, and its n -th term is q definitionally.*

```
def constant (q : MulRat) : MulCauchyStream where
  term := fun _ => q
```

```

cauchy := by
  intro level
  refine {one, ?_}
  intro m n hm hn
  exact MulRat.scaleWithin_refl level q

```

Proof. Given a level, take the anchor to be one, so that the tail is everything. Both terms are q , and `scaleWithin_refl` of `PrimeTensor/Scale.lean` applies at every level. The hypotheses `hm` and `hn` are discarded unused. $\square$

That reflexivity is unconditional matters here: had the ladder kept the tolerance parameter of `PrimeTensor/Order.lean`, this construction would have had to carry an admissibility hypothesis $1 < \delta$ into every constant stream, and from there into `offRat` and into every rational literal of the completion.

1.4 Asymptotic equivalence

Definition 1.13 (`MulAsymptotic`). Cauchy streams a and b are *asymptotic* when

$$\forall \ell \exists c \forall n : \text{AtOrAfter}(c, n) \longrightarrow \text{ScaleWithin}(\ell, a_n, b_n).$$

Note that the definition mentions only the term field: the Cauchy proofs carried by a and b are never used, here or in any of the three lemmas below. The relation would be well formed on raw streams; it is stated on the bundled structure because it is the bundled structure that gets quotiented.

Lemma 1.14 (`MulAsymptotic.refl` and `MulAsymptotic.symm`). *MulAsymptotic is reflexive and symmetric.*

Proof. For reflexivity take the anchor one and apply `scaleWithin_refl` termwise. For symmetry keep the anchor supplied by the hypothesis and apply `scaleWithin_symm` termwise; the level is not changed in either case. $\square$

Theorem 1.15 (`MulAsymptotic.trans`). *MulAsymptotic is transitive.*

```

@[trans] theorem trans {a b c : MulCauchyStream}
  (hab : MulAsymptotic a b) (hbc : MulAsymptotic b c) :
  MulAsymptotic a c := by
  intro level
  obtain ⟨abAnchor, habTail⟩ := hab (.succ level)
  obtain ⟨bcAnchor, hbcTail⟩ := hbc (.succ level)
  let anchor := Depth.join abAnchor bcAnchor
  refine {anchor, ?_}
  intro n hn
  have habN : Depth.AtOrAfter abAnchor n :=
    Depth.atOrAfter_trans (Depth.left_atOrAfter abAnchor bcAnchor) hn
  have hbcN : Depth.AtOrAfter bcAnchor n :=
    Depth.atOrAfter_trans (Depth.right_atOrAfter abAnchor bcAnchor) hn
  exact MulRat.scaleWithin_comp (habTail n habN) (hbcTail n hbcN)

```

Proof. Fix a level ℓ . Instantiate *both* hypotheses not at ℓ but at $\text{succ } \ell$, obtaining anchors a_{ab} and a_{bc} beyond which each leg is close at that finer level. Let $c^* = \text{join}(a_{ab}, a_{bc})$ and take it as the anchor for ℓ .

For n at or after c^* : by Lemma 1.7, c^* is at or after each of a_{ab} and a_{bc} , so Lemma 1.4 places n in both tails. Both hypotheses therefore apply at n , giving $\text{ScaleWithin}(\text{succ } \ell, a_n, b_n)$ and $\text{ScaleWithin}(\text{succ } \ell, b_n, c_n)$, and scaleWithin_comp of `PrimeTensor/Scale/Laws.lean` composes them into $\text{ScaleWithin}(\ell, a_n, c_n)$. $\square$

Design note 1.16 (The $\varepsilon/2$ argument, without $\varepsilon/2$). The classical proof of this statement asks each hypothesis for $\varepsilon/2$, takes $N = \max(N_1, N_2)$, and adds the two halves with the triangle inequality. The proof above has the same three moves and none of the arithmetic: it asks each hypothesis for $\text{succ } \ell$, takes join of the two anchors, and combines with the composition law.

Each replacement is forced by what the object language contains. Halving a tolerance would need division in the tolerance space; the tolerances here are $2^{1/2^k}$ and are not elements of MulRat at all, as `PrimeTensor/Scale.lean` recorded, so they cannot be divided — but the *index* of the tolerance can be raised by one, and by `PrimeTensor/Scale/Laws.lean` raising it by one is precisely halving. Taking a maximum would need an order on the index set with binary suprema; the index set is Depth , which carries neither until Definition 1.1 and Definition 1.6 supply exactly the two facts used. This is why the first half of this file exists.

The cost is that transitivity is proved by strengthening the request, so the hypotheses must be universally quantified over levels — which they are, and which is why MulAsymptotic could not have been stated at one fixed level.

Definition 1.17 (`MulAsymptotic.setoid`). The setoid on MulCauchyStream whose relation is MulAsymptotic and whose proof of equivalence is the triple assembled from Lemma 1.14 and Theorem 1.15.

1.5 The completion

Definition 1.18 (`MulReal` and its constructors).

$$\begin{aligned} \text{MulReal} &\equiv \text{Quotient}(\text{MulAsymptotic.setoid}), \\ \text{ofStream}(s) &\equiv [s], \\ \text{ofRat}(q) &\equiv \text{ofStream}(\text{constant}(q)), \\ \text{one} &\equiv \text{ofRat}(1), \end{aligned}$$

with one registered as the `One` instance of MulReal .

```
| abbrev MulReal := Quotient MulAsymptotic.setoid
```

Remark 1.19 (What the carrier is, and what it is not yet). MulReal is a type with a distinguished point and a map in from MulRat , and that is the whole of its structure at the end of this file. There is no multiplication, no inverse, no order, and no lemma relating ofRat to the multiplication of MulRat ; nor is ofRat shown injective, which is the statement that distinct rationals stay distinct in the completion and needs an argument that two constant streams asymptotic at every level are equal. Even one is so far only a point: nothing says it is a unit, there being nothing for it to be a unit for.

The reason the file stops here is that each of those requires the same ingredient — a proof that the operation in question respects `MulAsymptotic`, which is a fresh $\varepsilon/2$ -style argument per operation — and the machinery for those arguments is exactly what has just been assembled.

Remark 1.20 (The Cauchy field is inert so far). No proof in this file reads the cauchy field of a stream: the three equivalence lemmas use only `term`, and the constant stream supplies its proof without anyone consuming one. The field is nonetheless load-bearing for what follows, since a quotient of arbitrary streams by eventual agreement is not a completion of anything. It first does work when an operation has to be shown to send Cauchy streams to Cauchy streams.

Summary of the correspondence

Lean declaration	Kind	Here
<code>Depth.AtOrAfter</code>	inductive	Def. 1.1
<code>Depth.one_atOrAfter</code>	theorem	Lem. 1.2
<code>Depth.succ_atOrAfter</code>	theorem	Lem. 1.3
<code>Depth.atOrAfter_trans</code>	theorem	Lem. 1.4
<code>Depth.join</code>	definition	Def. 1.6
<code>Depth.left_atOrAfter</code>	theorem	Lem. 1.7
<code>Depth.right_atOrAfter</code>	theorem	Lem. 1.7
<code>MulStream</code>	abbrev	Def. 1.9
<code>IsMulCauchy</code>	definition	Def. 1.9
<code>MulCauchyStream</code>	structure	Def. 1.11
<code>MulCauchyStream.constant</code>	definition	Lem. 1.12
<code>MulCauchyStream.constant_term</code>	simp thm	Lem. 1.12
<code>MulAsymptotic</code>	definition	Def. 1.13
<code>MulAsymptotic.refl</code>	theorem	Lem. 1.14
<code>MulAsymptotic.symm</code>	theorem	Lem. 1.14
<code>MulAsymptotic.trans</code>	theorem	Thm. 1.15
<code>MulAsymptotic.setoid</code>	definition	Def. 1.17
<code>MulReal</code>	abbrev	Def. 1.18
<code>MulReal.ofStream</code>	definition	Def. 1.18
<code>MulReal.ofRat</code>	definition	Def. 1.18
<code>MulReal.one</code>	definition	Def. 1.18
<code>One MulReal</code>	instance	Def. 1.18

Every declaration of `PrimeTensor/Completion.lean` appears above, together with the two constructors here and later of `AtOrAfter` and the two fields `term` and `cauchy` of `MulCauchyStream`. The file contains no `sorry`, no `axiom`, and no unproved named proposition. Remarks 1.5, 1.8, 1.19 and 1.20 are stated for the reader and are not declarations of the file.